

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab# 04)

Use of Venus Simulator for Register Level Execution of RISC-V Arithmetic and Memory Instructions

Student Name: Abdulmoeiz Roll No: 24i-6510 Section: CE A

Date performed: 8 Sept, 2026

MARKS AWARDED: / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

In this task, we perform basic arithmetic operations in RISC-V using $A = 5$ and $B = 4$. We calculate addition, subtraction, multiplication, and square of A using different registers. The program is run in Venus, and the instruction fields are also represented in decimal form.

Code:

```
li s0, 5      # A = 5
li s1, 4      # B = 4

add s2, s0, s1 # C = A + B
sub s3, s0, s1 # D = A - B
mul s4, s0, s1 # E = A * B
mul s5, s0, s0 # F = A^2
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00500413	addi x8 x0 5	li s0, 5 # A=5
0x4	0x00400493	addi x9 x0 4	li s1, 4 # B=4
0x8	0x00940933	add x18 x8 x9	add s2, s0, s1 # C=A+B
0xc	0x409409B3	sub x19 x8 x9	sub s3, s0, s1 # D=A-B
0x10	0x02940A33	mul x20 x8 x9	mul s4, s0, s1 # E=A*B
0x14	0x02840AB3	mul x21 x8 x8	mul s5, s0, s0 # F=A^2

Task 2:

Description:
This task involves converting the given C expression into equivalent RISC-V assembly instructions. Different registers are assigned to the variables so that the required calculations can be performed step by step. The intermediate results are then combined according to the original expression to obtain the final value of m.
Code:
<pre>#(a - c) + (d + e) * f #s0 = a(3), s1 = c(1), s2 = d(2), s3 = e(2), s4 = f(5) li s0, 3 li s1, 1 li s2, 2 li s3, 2 li s4, 5 sub t1, s0, s1 add t2, s2, s3 mul t3, t2, s4 add s6, t1, t3</pre>
Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x00300413	addi x8 x0 3	li s0, 3
0x4	0x00100493	addi x9 x0 1	li s1, 1
0x8	0x00200913	addi x18 x0 2	li s2, 2
0xc	0x00200993	addi x19 x0 2	li s3, 2
0x10	0x00500A13	addi x20 x0 5	li s4, 5
0x14	0x40940333	sub x6 x8 x9	sub t1, s0, s1
0x18	0x013903B3	add x7 x18 x19	add t2, s2, s3
0x1c	0x03438E33	mul x28 x7 x20	mul t3, t2, s4
0x20	0x01C30B33	add x22 x6 x28	add s6, t1, t3

Task 3:

Description:
The purpose of this task is to perform three basic bitwise operations on the given hexadecimal numbers using RISC-V assembly language. The values are loaded into registers and the AND, OR, and XOR instructions are applied to them separately. The resulting register values are then observed in Venus to verify that the operations have been performed correctly.
Code:
<pre>li t0, 0xffff # t0 = 0xffff li t1, 0xf0f # t1 = 0xf0f and t2, t0, t1 or t3, t0, t1 xor t4, t0, t1</pre>
Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x000012B7	lui x5 1	li t0, 0xfff # t0 = 0xfff
0x4	0xFFFF28293	addi x5 x5 -1	li t0, 0xfff # t0 = 0xfff
0x8	0x00001337	lui x6 1	li t1, 0xf0f # t1 = 0xf0f
0xc	0xF0F30313	addi x6 x6 -241	li t1, 0xf0f # t1 = 0xf0f
0x10	0x0062F3B3	and x7 x5 x6	and t2, t0, t1 # t2 = t0 AND t1 → expect 0xf0f... wait, 0xf0f
0x14	0x0062EE33	or x28 x5 x6	or t3, t0, t1 # t3 = t0 OR t1 → expect 0xfff
0x18	0x0062CEB3	xor x29 x5 x6	xor t4, t0, t1 # t4 = t0 XOR t1 → expect 0xf0

Task 4:

Description:
In this task, the provided RISC-V assembly program is tested in the Venus simulator and compared with the given C implementation for calculating the sum of an integer array. The registers and program execution are observed to understand how the loop accesses each array element and adds it to the result. Any syntax or logical problems found during execution are corrected so that the assembly program produces the expected result.
Code:
<pre>.data arr: .word 3, 5, 2 .text .globl main main: la a0, arr</pre>

```
li a1, 3
```

```
jal arraysum
```

```
li a7, 1
```

```
ecall
```

```
li a7, 10
```

```
ecall
```

```
arraysum:
```

```
li t0, 0
```

```
li t1, 0
```

```
1:
```

```
bge t1, a1, 2f
```

```
slli t2, t1, 2
```

```
add t2, a0, t2
```

```
lw t2, 0(t2)
```

```
add t0, t0, t2
```

```
addi t1, t1, 1
```

```
j 1b
```

```
2:
```

```
mv a0, t0
```

```
ret
```

Output:

PC	Machine Code	Basic Code	Original Code
0x0	0x10000517	auipc x10 65536	la a0, arr
0x4	0x00050513	addi x10 x10 0	la a0, arr
0x8	0x00300593	addi x11 x0 3	li a1, 3
0xc	0x014000EF	jal x1 20	jal arraysum
0x10	0x00100893	addi x17 x0 1	li a7, 1
0x14	0x00000073	ecall	ecall
0x18	0x00A00893	addi x17 x0 10	li a7, 10
0x1c	0x00000073	ecall	ecall
0x20	0x00000293	addi x5 x0 0	li t0, 0
0x24	0x00000313	addi x6 x0 0	li t1, 0

[Copy!](#)
[Download!](#)
[Clear!](#)

console output

Integer (K) Floating (F)

zero

0

ra (x1)

16

sp (x2)

2147483612

gp (x3)

268435456

tp (x4)

0

t0 (x5)

10

t1 (x6)

3

t2 (x7)

2

s0 (x8)

0

s1 (x9)

0

a0 (x10)

10

a1 (x11)

3

Task 5:

Description:

This task focuses on creating the multiplication table of two using RISC-V assembly language. The table values are first stored in the data section using the .data directive. These values are then loaded into separate registers with the lw instruction so their contents can be checked and used during program execution in Venus.

Code:

```
.data

table:
.word 2
.word 4
.word 6
.word 8
```

```
.word 10
.word 12
.word 14
.word 16
.word 18
.word 20

.text
la t0, table

lw t1, 0(t0)    # t1 = 2
lw t2, 4(t0)    # t2 = 4
lw t3, 8(t0)    # t3 = 6
lw t4, 12(t0)   # t4 = 8
lw t5, 16(t0)   # t5 = 10
lw t6, 20(t0)   # t6 = 12
```

Output:

Venus
Editor
Simulator
Chocopy

Run
Step
Prev
Reset
Dump
Trace
Re-assemble from Editor

PC	Machine Code	Basic Code	Original Code
0x0	0x10000297	auipc x5 65536	la t0, table
0x4	0x00028293	addi x5 x5 0	la t0, table
0x8	0x0002A303	lw x6 0(x5)	lw t1, 0(t0) # t1 = 2
0xc	0x0042A383	lw x7 4(x5)	lw t2, 4(t0) # t2 = 4
0x10	0x0082AE03	lw x28 8(x5)	lw t3, 8(t0) # t3 = 6
0x14	0x00C2AE83	lw x29 12(x5)	lw t4, 12(t0) # t4 = 8
0x18	0x0102AE03	lw x30 16(x5)	lw t5, 16(t0) # t5 = 10
0x1c	0x0142AE03	lw x31 20(x5)	lw t6, 20(t0) # t6 = 12

s9 (x25) 0
s10 (x26) 0
s11 (x27) 0
t3 (x28) 6
t4 (x29) 8
t5 (x30) 10
t6 (x31) 12

Copy!
Download!
Clear!